

Lab 6a — Intermediate SQL Practice & Analytics

Module I & II · Medium Difficulty SQL Practice · Duration: 2 hours

Tools: PostgreSQL / SQLite · Deliverables: queries.sql, lab06a_report.md

Objectives

This lab provides medium-difficulty practice on core SQL querying, multi-table aggregations, window functions, correlated subqueries, and database views using the **StreamFlix** streaming analytics platform database:

1. **Multi-table Aggregations & Conditional Logic** (GROUP BY, HAVING, CASE).
 2. **Window Ranking Functions** (DENSE_RANK(), PARTITION BY).
 3. **Correlated Subqueries & Anti-Joins** (NOT EXISTS, subquery filtering).
 4. **Lead/Lag Window Functions** (LEAD(), JULIANDAY date arithmetic).
 5. **Database Views & Outer Joins** (CREATE VIEW, LEFT JOIN).
-

Database Schema (StreamFlix)

StreamFlix operates an online media streaming service. The database schema consists of 5 core tables:

```
users(user_id PK, name, email, country, signup_date, subscription_tier ['Basic','Standard','Premium'])
content(content_id PK, title, genre, release_year, duration_minutes, type ['Movie','Series'])
subscriptions(sub_id PK, user_id FK, start_date, end_date, monthly_amount, payment_status ['active','cancelled'])
watch_history(watch_id PK, user_id FK, content_id FK, watch_date, watch_time_minutes, device ['Mobile','TV'])
ratings(rating_id PK, user_id FK, content_id FK, score [1-5], rating_date)
```

Setup (5 min)

1. Load the schema and sample dataset into your database environment (SQLite or PostgreSQL):

```
sqlite3 lab6a.db < data/schema.sql
sqlite3 lab6a.db < data/data.sql
```

2. Verify that all 5 tables are populated:

```
SELECT table_name FROM information_schema.tables WHERE table_schema='public'; -- PostgreSQL
-- OR --
.tables -- SQLite
```

Part 1 — Task 1: User Viewing Profile & Binge Engagement (Aggregations & Grouping)

Business Context: StreamFlix wants to analyze viewing engagement levels across its user base and categorize users by total watching volume.

Task Requirements: 1. Retrieve `user_id`, `name`, and `subscription_tier`. 2. Compute `total_watch_time_mins`: the sum of `watch_time_minutes` for each user. 3. Compute `titles_watched_count`: the count of distinct `content_ids` watched by the user. 4. Compute `avg_session_duration`: average watch time per session, rounded to 1 decimal place. 5. Assign an `engagement_category`: - 'Power Viewer' if `total_watch_time_mins` ≥ 350 - 'Regular Viewer' if $150 \leq \text{total_watch_time_mins} < 350$ -

'Light Viewer' if `total_watch_time_mins < 150`. 6. Filter results using `HAVING` to include only users who have watched **at least 2 distinct titles**. 7. Order results by `total_watch_time_mins DESC`.

Part 2 — Task 2: Top Genre Rankings per Subscription Tier (Window Ranking)

Business Context: The content acquisition team needs to identify the top 2 preferred viewing genres for each subscription tier (`Basic`, `Standard`, `Premium`).

Task Requirements: 1. Aggregate the total watch time (`SUM(watch_time_minutes)`) for each combination of `subscription_tier` and `genre`. 2. Rank genres within each subscription tier using `DENSE_RANK() OVER (PARTITION BY subscription_tier ORDER BY total_watch_time DESC)`. 3. Filter output to show only the **top 2 ranked genres per subscription tier**. 4. Order results by `subscription_tier ASC` and `genre_rank ASC`.

Hint: Use a Common Table Expression (CTE) to aggregate viewing time before applying `DENSE_RANK()`.

Part 3 — Task 3: Inactive Active Subscriber Identification (Correlated Subqueries & Anti-Joins)

Business Context: The customer retention team wants to target paying subscribers who are currently inactive but had high historical ratings, offering them tailored content recommendations.

Task Requirements: 1. Select users who currently have an **active** subscription (`payment_status = 'active'`). 2. Filter for users who have **NOT watched any content in the 30 days prior to 2026-03-01** (i.e., no watch activity on or after `2026-01-30`). Use a correlated `NOT EXISTS` subquery. 3. Filter for users whose average rating score given across historical content is ≥ 4.0 . 4. Display `user_id`, `name`, `email`, `subscription_tier`, `last_watch_date` (`MAX(watch_date)`), and `avg_rating_given` (rounded to 2 decimal places). 5. Order by `last_watch_date ASC`.

Part 4 — Task 4: User Session Continuity & Gap Analysis (Lead Window Functions)

Business Context: Product designers want to analyze viewing frequency and flag long periods of user inactivity between watch sessions.

Task Requirements: 1. For every watch session of a user, retrieve `watch_id`, `user_id`, `name`, and `watch_date`. 2. Compute `next_watch_date`: the date of the user's next watch session using `LEAD(watch_date) OVER (PARTITION BY user_id ORDER BY watch_date)`. 3. Calculate `days_until_next_session` (difference in days between `next_watch_date` and `watch_date`). 4. Output a `session_gap_flag`: - 'LONG INACTIVITY BREAK' if `days_until_next_session >= 14` - 'NORMAL' otherwise. 5. Exclude the user's final watch session (where `next_watch_date IS NULL`). 6. Order results by `user_id ASC` and `watch_date ASC`.

Part 5 — Task 5: Content Catalog Performance & View Creation (Database Views & Outer Joins)

Business Context: Executive management wants a central analytics view for content performance metrics across all titles released in **2024 or later**.

Task Requirements: 1. Create a database View named `v_content_analytics`. 2. Select all content released in **2024 or later** (`release_year >= 2024`). 3. For each content item, include: - `content_id`, `title`,

genre, release_year, type. - total_views: Count of watch sessions (must return 0 for unwatched titles using LEFT JOIN). - total_watch_hours: SUM(watch_time_minutes) / 60.0, rounded to 2 decimal places (return 0.0 if unwatched). - avg_user_rating: AVG(score), rounded to 2 decimal places (NULL if no ratings exist). - rating_count: Count of ratings submitted. 4. Execute `SELECT * FROM v_content_analytics ORDER BY total_watch_hours DESC`; to verify your view.

Submission Checklist

Your submission folder `lab06a/` must contain: - **queries.sql** — Containing your tested SQL queries for Tasks 1–5, clearly commented. - **lab06a_report.md** — Short report containing your output tables for Tasks 1–5 and a 2-sentence explanation of the SQL mechanics for each.